SUBJECT :- MCAD
ALL UNIT GTU IMP QUE ANS BY SSD

UNIT 1

Q1. Write a short note on Mobile Operating Systems

A Mobile Operating System (Mobile OS) is software that runs on mobile devices like smartphones and tablets. It controls the hardware and allows users to run applications.

Key Points:

- Acts as an interface between user and device hardware
- Manages memory, processor, and applications
- Provides user interface (touch screen, icons, menus)
- Supports communication features like calling, messaging, and internet
- Ensures security and data protection

Examples:
Android
iOS

Q2. Write a short note on Android

Android is an open-source mobile operating system developed by Google. It is designed mainly for touchscreen devices such as smartphones and tablets, but is also used in smart TVs, wearables, and automobiles.

Introduction
Android is based on the Linux Kernel, which provides core system functionalities like memory management, process management, and hardware interaction. Because it is open-source, developers and manufacturers can modify and customize it according to their needs.

History of Android
Android Inc. was founded in 2003
Acquired by Google in 2005
First Android version launched in 2008
Since then, many versions have been released with improved features

Features of Android
1.Open Source
2.User-Friendly Interface
3.Multitasking
4.Connectivity
5.Customization

6.Google Services Integration
7.Large App Ecosystem
8.Security

Android Architecture
Android architecture consists of the following layers:

1. Linux Kernel – Core layer managing hardware
2. Libraries – Provides essential functionalities (SQLite, WebKit, etc.)
3. Android Runtime – Executes applications (ART/Dalvik)
4. Application Framework – Provides APIs for developers
5. Applications – User-installed apps

Advantages of Android

- Free and open-source
- Wide range of devices available
- High customization
- Large developer community
- Supports multiple programming languages

Disadvantages of Android

- Security issues compared to some other OS
- Fragmentation (many versions and devices)
- Performance may vary depending on device

Q3. Explain all features of Android

Android provides many powerful features that make it one of the most popular mobile operating systems. It is developed by Google.
Main Features of Android

1. Open Source
   - Android is an open-source platform, which means developers can access and modify the source code. This allows manufacturers to customize Android for their devices.

2. User-Friendly Interface
   - Android provides an easy-to-use interface with touch-based navigation like tapping, swiping, and pinching. It supports attractive layouts and designs.

3. Multitasking
   - Android supports multitasking, which means users can run multiple applications at the same time, such as listening to music while browsing the internet.

4. Connectivity
Android supports various connectivity options:

- Wi-Fi
- Bluetooth
- Mobile networks (3G, 4G, 5G)
- NFC (Near Field Communication)

5. Customization
- Users can customize their devices by:
- Changing themes and wallpapers
- Adding widgets
- Rearranging icons and layouts

6. Google Services Integration
Android devices come with built-in services like:
- Gmail
- Google Maps
- Google Drive
- YouTube

7. Large App Ecosystem
- Android has millions of apps available on the Google Play Store, including games, tools, and educational apps.

8. Security
Android provides security features such as:
- App permissions
- Data encryption
- Screen lock (PIN, pattern, fingerprint)

9. Multiple Language Support
- Android supports many languages, making it usable worldwide.

10. Hardware Support
- Android supports a wide range of hardware like cameras, sensors, GPS, and fingerprint scanners.

Q4. Enlist all versions of Android

Android has released many versions since its launch in 2008. Earlier versions were named after desserts, while newer versions use only numbers. Android is developed by Google.

List of Android Versions
1.Android 1.0 (2008)
2.Android 1.1 (2009)
3. Android 1.5 – Cupcake

4. Android 1.6 – Donut
5. Android 2.0 / 2.1 – Eclair
6. Android 2.2 – Froyo (Frozen Yogurt)
7. Android 2.3 – Gingerbread
8. Android 3.x – Honeycomb (Tablet version)
9. Android 4.0 – Ice Cream Sandwich
10. Android 4.1 – 4.3 – Jelly Bean
11. Android 4.4 – KitKat
12. Android 5.0 / 5.1 – Lollipop
13. Android 6.0 – Marshmallow
14. Android 7.0 / 7.1 – Nougat
15. Android 8.0 / 8.1 – Oreo
16. Android 9.0 – Pie
17. Android 10
18. Android 11
19. Android 12
20. Android 13
21. Android 14

Q5. Comparison between Android and iOS

Android is developed by Google, while iOS is developed by Apple Inc..

Difference between Android and iOS

Android
iOS
1. Developer
    - Developed by Google
    - Developed by Apple Inc.
2. Source Type
    - Open-source operating system
    - Closed-source operating system
3. Device Support
    - Available on many brands (Samsung, Xiaomi, etc.)
    - Available only on Apple devices
4. Customization
    - Highly customizable (themes, widgets, launchers)
    - Limited customization options
5. App Store
    - Uses Google Play Store and third-party sources
    - Uses only Apple App Store
6. Programming Languages
    - Java, Kotlin

- Swift, Objective-C

7. Security Level
   - Moderate (more open, so more risks)
   - High (strict control and policies)
8. User Interface
   - Flexible and changeable
   - Simple and consistent
9. Price Range
   - Available in low to high budget phones
   - Mostly premium and expensive devices
10. Software Updates
    - Depends on manufacturer and device
    - Regular and directly from Apple

Q6. Give introduction to Android Studio

Android Studio is the official Integrated Development Environment (IDE) used for developing Android applications. It is developed by Google and is based on IntelliJ IDEA.

Introduction
Android Studio provides all the tools required to design, develop, test, and debug Android apps in one place. It supports programming languages like Java and Kotlin.

Features of Android Studio

1.Code Editor
   - Provides smart code completion, syntax highlighting, and error detection.

2.Layout Editor
   - Allows designing UI using drag-and-drop components.

3.Emulator
   - Built-in virtual device to test applications without a physical device.

4.Debugging Tools
   - Helps to find and fix errors in the application.

5.Gradle Build System
   - Used for project building and dependency management.

6.APK Analyzer
   - Helps to reduce app size and analyze APK files.

7.Support for Multiple Languages
   - Supports Java, Kotlin, and C++.

Advantages
- Easy to use interface
- Official tool for Android development
- Powerful testing and debugging features

- Regular updates from Google

Q7. How to run apps on Emulator / Physical Device

Android applications can be tested either on an Emulator (virtual device) or a Physical Device (real mobile phone) using Android Studio.

1. Running App on Emulator
An Emulator is a virtual Android device that runs on a computer.

Steps:
1. Open Android Studio
2. Create a project or open existing project
3. Go to AVD Manager (Android Virtual Device)
4. Click on Create Virtual Device
5. Select device (e.g., Pixel) and Android version
6. Start the emulator
7. Click Run ▶️ button
8. App will install and run on emulator

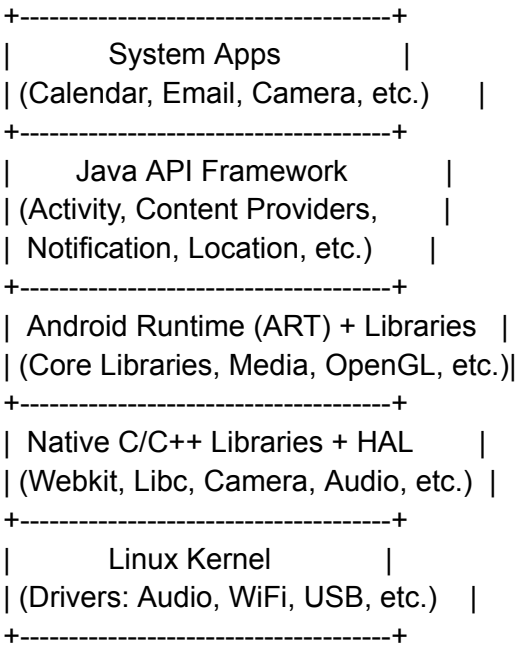
2. Running App on Physical Device
A Physical Device is a real Android mobile phone.

Steps:
1. Enable Developer Options in phone
2. Turn on USB Debugging
3. Connect phone to computer using USB cable
4. Allow permission on phone (if asked)
5. Open Android Studio
6. Select your device from device list
7. Click Run ▶️ button
8. App will install and run on your phone

UNIT 2

Q1. Explain Android Architecture with neat figure

Android architecture is a layered structure that shows how different components of Android work together. It is developed by Google.
Neat Figure of Android Architecture



Explanation of Layers

1. Linux Kernel

- Lowest layer of Android
- Manages hardware through drivers
- Includes drivers for Audio, Camera, Bluetooth, WiFi, USB
- Handles process management, memory, and power management
- Uses Binder (IPC) for communication

2. Hardware Abstraction Layer (HAL)

- Connects hardware and software
- Provides standard interface for hardware components
- Examples: Audio, Camera, Sensors, Bluetooth

3. Native C/C++ Libraries

- Written in C/C++
- Provide core functionalities
- Examples:
  - WebKit (browser engine)
  - OpenGL ES (graphics)
  - Libc (standard C library)
  - OpenMAX AL (media support)

4. Android Runtime (ART)

- Executes Android applications
- Includes Core Libraries

- Each app runs in separate process for security

5. Java API Framework

- Provides APIs for developers
- Includes components like:
    - Activity Manager
    - Content Providers
    - Notification Manager
    - Location Manager
    - View System (UI)

6. System Applications

- Top layer
- Includes apps like Calendar, Email, Camera
- Built using framework APIs

Q2. Explain Activity and its Life Cycle

What is Activity?

An Activity is one of the main components of an Android application. It represents a single screen with a user interface where the user can interact.

Examples:

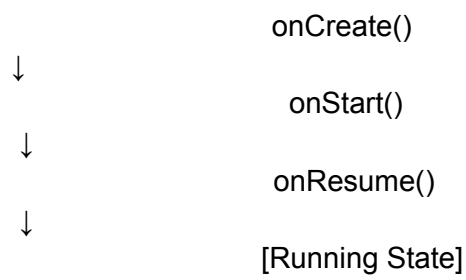
- Login screen
- Home screen
- Settings screen

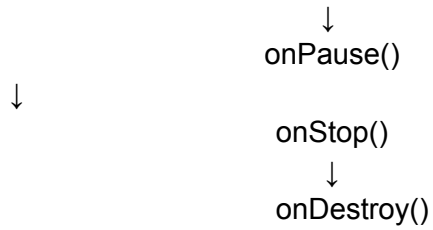
Every Android app usually contains multiple activities, and each activity is responsible for a specific task. Activities are managed by the Android system and are developed using tools like Android Studio.

Activity Lifecycle

The Activity Lifecycle is the sequence of states an activity goes through during its lifetime (from creation to destruction). It is controlled by the Android system.

Neat Diagram of Activity Lifecycle





Explanation of Lifecycle Methods

1.onCreate() – Activity is created
2.onStart() – Activity becomes visible
3.onResume() – Activity starts running
4.onPause() – Activity is partially hidden
5.onStop() – Activity is not visible
6.onDestroy() – Activity is destroyed

Additional Concept (Back Navigation)

- When user presses Back button, activity goes through onPause() → onStop() → onDestroy()
- When user returns, activity may restart again

Importance of Activity Lifecycle

- Helps in managing memory efficiently
- Prevents app crashes
- Ensures smooth user experience
- Saves and restores app data properly

Activity Lifecycle Code Example

Java

```
package com.example.lifecycle;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Log.d("Lifecycle", "onCreate called");
    }

    @Override
    protected void onStart() {
```

```
        super.onStart();
        Log.d("Lifecycle", "onStart called");
    }

    @Override
    protected void onResume() {
        super.onResume();
        Log.d("Lifecycle", "onResume called");
    }

    @Override
    protected void onPause() {
        super.onPause();
        Log.d("Lifecycle", "onPause called");
    }

    @Override
    protected void onStop() {
        super.onStop();
        Log.d("Lifecycle", "onStop called");
    }

    @Override
    protected void onDestroy() {
        super.onDestroy();
        Log.d("Lifecycle", "onDestroy called");
    }
}
```

Q3. Explain Services and its Life Cycle

What is Service?

A Service in Android is a component that runs in the background without a user interface. It is used to perform long-running operations such as playing music, downloading files, or fetching data from the internet.

Services are an important part of Android apps developed using Android Studio.

Types of Services

1.Started Service

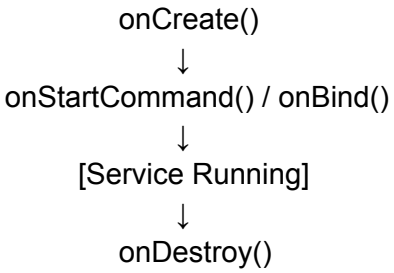
- Started using startService()
- Runs until it is stopped manually

2.Bound Service

- Allows other components to bind using bindService()
- Runs as long as it is bound

Service Lifecycle

Neat Diagram of Service Lifecycle



Explanation of Lifecycle Methods

1. onCreate()
   - Called when the service is first created
   - Used for initialization

2. onStartCommand()
   - Called when service is started using startService()
   - Service begins its work in background

3. onBind()
   - Called when a component binds to the service
   - Returns communication interface

4. onDestroy()
   - Called when service is stopped
   - Used to clean up resources

Advantages of Services
- Performs background tasks
- Improves user experience
- Allows multitasking

Example Uses
- Music player app
- File download service
- Background data sync

Q4. What is Broadcast Receiver?

Definition
A Broadcast Receiver is an Android component that is used to receive and respond to system-wide broadcast messages sent by the Android system or other applications.
These broadcasts inform apps about events happening in the device.

Key Features
- Works in the background (no user interface)
- Responds to system events
- Lightweight component (runs for short time)
- Registered in AndroidManifest.xml or dynamically in code

Common Examples of Broadcasts
- Battery low or battery full
- Incoming SMS
- Network connectivity change
- Device boot completed

How Broadcast Receiver Works
- System or app sends a broadcast message
- Broadcast Receiver receives the message
- Executes required action (e.g., show notification)

Types of Broadcasts
1. Normal Broadcast – Sent to all receivers
2. Ordered Broadcast – Sent one receiver at a time
3. Sticky Broadcast – Remains after broadcast (deprecated)

Q5. What is Content Provider?

Definition
A Content Provider is an Android component that is used to manage and share data between different applications in a secure way.
It allows one app to access data of another app using a standard interface.

Purpose of Content Provider
- To store and retrieve data
- To share data between apps
- To provide secure access to data

Key Features
- Provides data in structured format (like database)
- Uses URI (Uniform Resource Identifier) to identify data

Supports CRUD operations:

- Insert
- Read (Query)
- Update
- Delete

Ensures data security using permissions

How Content Provider Works

- Data is stored in database or files
- Content Provider manages this data
- Other apps access data using URI
- Operations are performed using ContentResolver

Common Methods

Java

- insert()
- query()
- update()
- delete()

Example

- Contacts app sharing contact data
- Media files (images, videos) sharing

Advantages

- Secure data sharing
- Centralized data management
- Easy access using standard APIs

Q6. Write a short note on Android Manifest

Introduction

Android Manifest is a configuration file of an Android app
It is named AndroidManifest.xml and stores app information
It is automatically created in Android Studio

Purpose of Android Manifest

Provides information about the application to the Android system
Declares all components of the app
Defines permissions required by the app

Main Contents of Manifest File

1.Package Name

- Unique identifier of the application

2.Application Components
- Activities
- Services
- Broadcast Receivers
- Content Providers

3.Permissions
- Internet access
- Camera usage
- Location access

4.Intent Filters
- Defines how components respond to intents

5.Minimum SDK Version
- Specifies minimum Android version required

Basic Structure of AndroidManifest.xml

XML

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.app">

    <application
        android:allowBackup="true"
        android:label="MyApp">

        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>

    </application>

</manifest>
```

Importance
- Mandatory file for every Android app
- Helps Android system understand app structure
- Controls app permissions and security

UNIT 3

Q1. Define Layouts. Explain types of layouts

Definition of Layout
In Android, a Layout is a container used to arrange and organize UI components (views and widgets) on the screen.
Layouts define the structure, position, and appearance of elements in an Android application. They are usually designed using XML in tools like Android Studio.

Types of Layouts

1. Linear Layout
   - Arranges elements in one direction (horizontal or vertical)
   - Uses orientation attribute
   - Simple and easy to use
   - Example: Vertical list of buttons

2. Relative Layout
   - Positions elements relative to each other or parent
   - Example: place button below a text view
   - More flexible than Linear Layout

3. Constraint Layout
   - Modern and flexible layout
   - Uses constraints to define position
   - Reduces nesting → better performance

4. Grid Layout
   - Arranges elements in rows and columns
   - Useful for table-like structure

Q2. Explain Views & Widgets used in Android

Introduction
In Android, Views and Widgets are the fundamental components used to build the User Interface (UI) of an application. They define how information is displayed and how users interact with the app. These components are designed and managed using tools like Android Studio.

What is a View?
A View is the basic building block of Android UI. It represents a rectangular area on the screen that can display content or handle user interaction.

Characteristics of Views:
- Occupies space on screen
- Can respond to user actions
- Can be customized (color, size, font, etc.)

Examples of Views:

- TextView – displays text
- ImageView – displays images
- Button – clickable component

What are Widgets?

Widgets are a type of views that are interactive and allow users to input data or perform actions. In simple words, all widgets are views, but not all views are widgets.

Common Views & Widgets in Android

1. TextView
   - Used to display text on the screen
   - Supports formatting, colors, and styles
2. EditText
   - Used to accept input from user
   - Commonly used in forms (login, signup)
3. Button
   - Used to perform actions when clicked
   - Works with event listeners like onClick()
4. ImageView
   - Displays images from resources or internet
5. ScrollView
   - Allows vertical scrolling of content
   - Useful when content is larger than screen
6. ListView
   - Displays a list of items in vertical format
   - Uses adapter to bind data
7. RecyclerView
   - Advanced version of ListView
   - Efficient and flexible
   - Reuses views using ViewHolder pattern

Difference between View and Widget

View

1. Basic UI element
2. Used for display
3. Example: TextView

Widget

1. Interactive UI element
2. Used for interaction
3. Example: Button

Importance of Views & Widgets

- Helps to design attractive UI
- Enables user interaction
- Makes application functional and user-friendly

Q3. Difference between ListView and RecyclerView

List view

1. ListView is an older component
2. ListView has lower performance
3. ListView has limited view reuse
4. ListView supports only vertical lists
5. ViewHolder is optional in ListView
6. ListView has limited animation support
7. ListView is less flexible

RecyclerView

1. RecyclerView is a newer and advanced component.
2. RecyclerView provides better performance.
3. RecyclerView uses ViewHolder for efficient reuse.
4. RecyclerView supports vertical, horizontal, and grid layouts.
5. mandatory in RecyclerView.
6. RecyclerView supports advanced animations.
7. RecyclerView is highly flexible.

Q4. Explain the concept of Event-driven Programming

Introduction

Event-driven programming is a programming paradigm in which the execution flow of a program is controlled by events. These events can be generated by user actions (like clicking a button) or by the system (like receiving a message).
It is widely used in Android application development using tools like Android Studio.

What is an Event?

An event is any action or occurrence that happens during the execution of a program.
Examples of events:

- Button click
- Touch on screen
- Key press
- Selecting an item from list
- Incoming notification

Working of Event-driven Programming

The working process includes the following steps:

1. Event occurs (user clicks a button)
2. Event Listener detects the event
3. Callback Method is executed
4. Required action is performed

Components of Event-driven Programming

1. Event Source
   - The object where the event occurs
   - Example: Button, EditText
2. Event Listener
   - Interface that listens for events
   - Example: OnClickListener
3. Callback Method
   - Method that runs when event happens
   - Example: onClick()

Example in Android

Java

```
Button btn = findViewById(R.id.button);

btn.setOnClickListener(new View.OnClickListener() {
   @Override
   public void onClick(View v) {
      // Action when button is clicked
      Toast.makeText(MainActivity.this, "Button Clicked", Toast.LENGTH_SHORT).show();
   }
});
```

Advantages of Event-driven Programming

- Makes applications interactive
- Improves user experience
- Easy handling of user actions
- Supports multitasking

Disadvantages

- Can be complex for large applications
- Hard to debug sometimes

Q5. Define: Event Listeners

Introduction
In Android, applications respond to user actions such as clicks, touches, and key presses. To handle these actions, Event Listeners are used. They are an important part of event-driven programming and are widely used in apps developed using Android Studio.

Definition
An Event Listener is an interface that listens for a specific event and triggers an action when that event occurs.

Purpose of Event Listeners
- To detect user interactions
- To handle different types of events
- To execute specific actions based on user input

Working of Event Listener
- An event occurs (e.g., button click)
- Event listener detects the event
- Callback method is called
- Required action is performed

Types of Event Listeners in Android

1. OnClickListener
   - Handles click events
   - Used with buttons
2. OnTouchListener
   - Handles touch events on screen
3. OnKeyListener
   - Handles keyboard key press events
4. OnFocusChangeListener
   - Handles focus change of views

Example Code
Java

```
Button btn = findViewById(R.id.button);

btn.setOnClickListener(new View.OnClickListener() {
   @Override
   public void onClick(View v) {
      // Action when button is clicked
      Toast.makeText(MainActivity.this, "Button Clicked", Toast.LENGTH_SHORT).show();
   }
```

});

Advantages of Event Listeners

- Makes applications interactive
- Easy handling of events
- Improves user experience
- Supports event-driven programming

Disadvantages

- Too many listeners can increase complexity
- Difficult to manage in large applications

Q6. Explain Callback Method in Event Handling

Introduction

In Android, applications need to respond to user actions such as clicking a button, touching the screen, or pressing a key. These actions are handled using callback methods, which are a key part of event-driven programming. They are commonly used in apps developed with Android Studio.

Definition

A Callback Method is a method that is automatically invoked (called back) by the Android system when a particular event occurs.

Purpose of Callback Methods

- To handle user interactions
- To execute specific code when an event occurs
- To connect event listeners with program logic

Working of Callback Method

The process works as follows:

1. An event occurs (e.g., user clicks a button)
2. An event listener detects the event
3. The system calls the corresponding callback method
4. The code inside the method is executed

Common Callback Methods in Android

1.onClick()

- Called when a button is clicked

2.onTouch()

- Called when the screen is touched

3.onKey()

- Called when a key is pressed

4.onFocusChange()

- Called when focus changes from one view to another

Example Code

Java

```
Button btn = findViewById(R.id.button);

btn.setOnClickListener(new View.OnClickListener() {
   @Override
   public void onClick(View v) {
      // Callback method executed on click
      Toast.makeText(MainActivity.this, "Button Clicked", Toast.LENGTH_SHORT).show();
   }
});
```

Advantages of Callback Methods

- Automatically executed by the system
- Simplifies event handling
- Makes applications interactive
- Improves user experience

Disadvantages

- Can be confusing for beginners
- Hard to manage in large applications

Q7. Difference between Event Listener and Callback Method

Event Listener

1. Event Listener is an interface
2. Event Listener listens for events
3. Event Listener detects the event
4. Event Listener must be implemented
5. Event Listener acts as a trigger
6. Event Listener is attached to UI components
7. Event Listener cannot execute action alone
8. Example:OnClickListener (Listener)

Callback Method

1. Callback Method is a method.
2. Callback Method handles events.
3. Callback Method performs action.

4. Callback Method is automatically called by system.
5. Callback Method acts as a response.
6. Callback Method is defined inside listener.
7. Callback Method contains actual logic.
8. Example:onClick() (Callback Method).

Q8. Explain Event Handling Methods: onClick(), onKey(), onTouch(), onFocusChange()

Introduction
In Android, applications interact with users through various actions like clicking buttons, touching the screen, or typing on the keyboard. These actions are handled using event handling methods, which are callback methods triggered when an event occurs. These are widely used in apps developed using Android Studio.

Event Handling Methods

1. onClick() Method
- This method is called when a user clicks on a view such as a button
- It is the most commonly used event handling method

Example:
Java

```
button.setOnClickListener(new View.OnClickListener() {
    public void onClick(View v) {
        // Action when button is clicked
    }
});
```

2. onKey() Method
- Called when a key is pressed, released, or held down
- Used for handling keyboard inputs

Example:
Java

```
view.setOnKeyListener(new View.OnKeyListener() {
    public boolean onKey(View v, int keyCode, KeyEvent event) {
        // Handle key press
        return true;
    }
});
```

3. onTouch() Method

- Called when the user touches the screen
- Can detect gestures like tap, swipe, long press

Example:
Java

```
view.setOnTouchListener(new View.OnTouchListener() {
    public boolean onTouch(View v, MotionEvent event) {
        // Handle touch event
        return true;
    }
});
```

4. onFocusChange() Method

- Called when a view gains or loses focus
- Useful for input validation and UI updates

Example:
Java

```
view.setOnFocusChangeListener(new View.OnFocusChangeListener() {
    public void onFocusChange(View v, boolean hasFocus) {
        // Handle focus change
    }
});
```

Importance of Event Handling Methods

- Makes applications interactive
- Handles user inputs effectively
- Improves user experience
- Supports event-driven programming

Q9. Explain item selection events using onItemSelected() (ListView)

Introduction

In Android, when a user selects an item from a list (like ListView or Spinner), an event is generated. This event is handled using the onItemSelected() method. It is commonly used in apps developed with Android Studio.

What is onItemSelected()?

onItemSelected() is a callback method that is called when a user selects an item from a list.

Working of onItemSelected()

1. User selects an item from ListView/Spinner

2. Event listener detects selection
3. onItemSelected() method is called
4. Selected item position and value are returned

Methods in OnItemSelectedListener

1.onItemSelected()
- Called when an item is selected

2.onNothingSelected()
- Called when no item is selected

Example Code
Java

```
Spinner spinner = findViewById(R.id.spinner);

spinner.setOnItemSelectedListener(new AdapterView.OnItemSelectedListener() {

    @Override
    public void onItemSelected(AdapterView<?> parent, View view, int position, long id) {
        String item = parent.getItemAtPosition(position).toString();
        Toast.makeText(MainActivity.this, "Selected: " + item, Toast.LENGTH_SHORT).show();
    }

    @Override
    public void onNothingSelected(AdapterView<?> parent) {
        // Do nothing
    }
});
```

Key Points
- Used for item selection events
- Works with ListView/Spinner
- Returns selected item and position
- Helps in user interaction

Q10. Define: Intents. Explain Implicit and Explicit Intents with example

Introduction
In Android, Intents are used to communicate between different components of an application such as Activities, Services, and Broadcast Receivers. Intents are an important part of Android app development using Android Studio.

Definition of Intent

An Intent is a messaging object used to request an action from another component in an Android application.

Types of Intents

1. Explicit Intent

- An Explicit Intent is used when we clearly specify the target component (activity) that we want to open.

Example:
Java

```
Intent i = new Intent(MainActivity.this, SecondActivity.class);
startActivity(i);
```

Use:

- Used to navigate between activities inside the same app

2. Implicit Intent

- An Implicit Intent is used when we do not specify the target component. Instead, we declare an action, and Android system finds suitable app to handle it.

Example:
Java

```
Intent i = new Intent(Intent.ACTION_VIEW);
i.setData(Uri.parse("https://www.google.com"));
startActivity(i);
```

Use:

- Used to open browser, dialer, maps, etc.

Q11. Difference between Explicit Intent and Implicit Intent

Explicit Intent

1. Explicit Intent directly specifies the target component
2. Explicit Intent is used within the same application
3. Explicit Intent clearly defines the destination activity
4. Explicit Intent provides full control to the developer
5. Explicit Intent is mainly used for navigation between activities
6. Explicit Intent uses class name of activity
7. Explicit Intent is more secure as it is predefined
8. Example:Open SecondActivity

Implicit Intent

1. Implicit Intent does not specify the target.
2. Implicit Intent is used to open other applications.
3. Implicit Intent lets the system choose suitable app.
4. Implicit Intent control is handled by Android system.
5. Implicit Intent is used for actions like opening browser, dialer, etc.
6. Implicit Intent uses action (e.g., ACTION_VIEW).
7. Implicit Intent depends on available apps.
8. Example:Open web browser

Q12. How to pass data between activities

Introduction
In Android applications, different screens are called activities. Many times, we need to transfer data from one activity to another (for example: username, email, or score). This is done using Intent in Android. It is an important concept in apps developed using Android Studio.

Definition
Passing data between activities means sending information from one activity (screen) to another activity using Intent objects.

Method Used
Android uses Intent Extras to pass data:
1. putExtra() → used to send data
2. getIntent() → used to receive Intent
3. getXXXExtra() → used to get data

Steps to Pass Data Between Activities

Step 1: Sending Data (First Activity)
Java

```
Intent intent = new Intent(MainActivity.this, SecondActivity.class);
intent.putExtra("name", "Rahul");
intent.putExtra("age", 21);
startActivity(intent);
```

Explanation:
- "name" and "age" are keys
- "Rahul" and 21 are values

Step 2: Receiving Data (Second Activity)

Java

```
Intent intent = getIntent();
String name = intent.getStringExtra("name");
int age = intent.getIntExtra("age", 0);
```

Explanation:

- Data is retrieved using same keys
- Default value is given for safety

Types of Data that can be passed

1. String
2. Integer
3. Boolean
4. Float
5. Array / Bundle

Example Use Cases

- Login page → Home page (username transfer)
- Registration form → Confirmation page
- Quiz app → Score display

Advantages

- Simple and fast communication
- Supports multiple data types
- Helps in navigation between screens
- Widely used in Android apps

Q13. Define: Toasts, Dialogs, and Menu with its Options, Context and Popup

Introduction

In Android, user interaction is improved using Toast, Dialog, and Menu components. These are used to show messages, take user input, and provide different options to users. They are commonly used in Android apps developed using Android Studio.

1.Toast

Definition

- A Toast is a small pop-up message that appears on the screen for a short duration and disappears automatically without any user action.

Features of Toast

- Non-interactive message
- Appears for short time
- Does not block user activity

- Used for simple notifications

Syntax / Example
Java

```
Toast.makeText(getApplicationContext(), "Data Saved Successfully",
Toast.LENGTH_SHORT).show();
```

Use Cases
- “Login Successful” message
- “Data Saved” confirmation
- Error notifications

2. Dialog
Definition
- A Dialog is a small popup window that appears on top of the current screen and requires user interaction before continuing.

Features of Dialog
- Interactive window
- Blocks background activity
- Used for confirmation or input
- Requires user action (Yes/No, OK/Cancel)

Types of Dialog
- Alert Dialog
- Progress Dialog
- Custom Dialog

Example
Java

```
AlertDialog.Builder builder = new AlertDialog.Builder(this);
builder.setTitle("Exit App");
builder.setMessage("Are you sure you want to exit?");
builder.setPositiveButton("Yes", null);
builder.setNegativeButton("No", null);
builder.show();
```

Use Cases
- Exit confirmation
- Delete confirmation
- Warning messages

3. Menu

Definition

- A Menu is a list of options provided to the user to perform different actions in an application.

Types of Menu

1. Options Menu
   - Displayed in the app bar (top of screen)
   - Contains main app actions
2. Context Menu
   - Appears on long press of a view
   - Depends on selected item
3. Popup Menu
   - Small floating menu near a view
   - Used for quick actions

Example (Menu XML)

XML

```
<menu xmlns:android="http://schemas.android.com/apk/res/android">
   <item android:title="Settings"/>
   <item android:title="Profile"/>
   <item android:title="Logout"/>
</menu>
```

Use Cases

- Settings option
- Profile actions
- App navigation options

UNIT 4

Q1. Explain Shared Preferences in Data Storage in Android

Introduction

In Android, Data Storage is used to save application data for later use. One of the simplest storage methods is Shared Preferences, which is mainly used for storing small and simple data. It is commonly used in applications developed using Android Studio.

Definition

Shared Preferences is a lightweight storage mechanism in Android used to store small amounts of data in key-value pair format. It stores primitive data types such as String, int, boolean, float, and long.

Features of Shared Preferences

- Stores data in key-value pair format
- Stores small and simple data only
- Data is private to the application
- Data remains saved even after closing the app
- Easy to implement and use
- Fast access to stored data

Modes of Shared Preferences

1.MODE_PRIVATE

- Data can only be accessed within the same application
- Most commonly used mode

2.MODE_APPEND

- Adds new data to existing file (rarely used)

Methods Used in Shared Preferences

- putString() → store string value
- putInt() → store integer value
- putBoolean() → store boolean value
- getString() → retrieve string value
- getInt() → retrieve integer value
- apply() → save data asynchronously
- commit() → save data synchronously

Working of Shared Preferences

Create SharedPreferences object
Create Editor object
Store data using key-value pairs
Apply or commit changes
Retrieve data when required

Example Code

Java

```
SharedPreferences sp = getSharedPreferences("UserData", MODE_PRIVATE);
SharedPreferences.Editor editor = sp.edit();

editor.putString("name", "Rahul");
```

```
editor.putInt("age", 20);
editor.putBoolean("isLoggedIn", true);

editor.apply();
```

Retrieving Data
Java

```
SharedPreferences sp = getSharedPreferences("UserData", MODE_PRIVATE);

String name = sp.getString("name", "");
int age = sp.getInt("age", 0);
boolean login = sp.getBoolean("isLoggedIn", false);
```

Advantages
- Very easy to use
- Fast data access
- Lightweight storage
- No database required for small data

Limitations
- Not suitable for large data
- Only supports primitive data types
- Not secure for sensitive data

Use Cases
- Saving login status
- User settings (dark mode, notifications)
- App preferences

Q2. Write a short note on SQLite Database

Introduction
In Android, data storage is an important feature used to store and manage application data. One of the most commonly used local databases is SQLite Database, which is used in apps developed using Android Studio.

Definition
SQLite Database is a lightweight, embedded, and open-source relational database management system used in Android to store structured data locally in the form of tables.

Features of SQLite
- Lightweight and fast database
- No server required (serverless database)

- Stores data in tables (rows and columns)
- Supports SQL (Structured Query Language)
- Fully embedded in Android system
- Works offline

Advantages of SQLite
- Easy to use in Android apps
- No installation required
- Secure local storage
- Efficient for small and medium data
- Fast data processing

Disadvantages
- Not suitable for large-scale applications
- Limited multi-user support
- No built-in user management system

Structure of SQLite
- Database → Collection of tables
- Table → Collection of rows and columns
- Row → Single record
- Column → Field/attribute

Example Use Cases
- Contact apps
- Notes apps
- Offline data storage apps
- Student management systems

Q3. Explain SQLiteOpenHelper Class in SQLite Database

Introduction
In Android, data can be stored locally using SQLite Database, which is a lightweight relational database. To manage this database properly (creation, updating, and version control), Android provides a helper class called SQLiteOpenHelper. It is widely used in applications developed using Android Studio.

Definition
SQLiteOpenHelper is a built-in Android class that helps developers to create, open, and manage SQLite databases efficiently. It also handles database version management like upgrade and downgrade.

Purpose of SQLiteOpenHelper
To create database automatically

To create tables inside database
To manage database version
To upgrade database structure when needed
To reduce manual database handling code

Constructor of SQLiteOpenHelper

Java

SQLiteOpenHelper(Context context, String name, SQLiteDatabase.CursorFactory factory, int version)

Parameters:

- context → Application context
- name → Database name
- factory → Cursor factory (usually null)
- version → Database version number

Important Methods

1. onCreate()
   - Called when database is created for the first time
   - Used to create tables and initial structure

Java

```
@Override
public void onCreate(SQLiteDatabase db) {
   db.execSQL("CREATE TABLE Student(id INTEGER PRIMARY KEY, name TEXT)");
}
```

2. onUpgrade()
   - Called when database version is changed
   - Used to update or modify database structure

Java

```
@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
   db.execSQL("DROP TABLE IF EXISTS Student");
   onCreate(db);
}
```

Working of SQLiteOpenHelper

- App requests database
- SQLiteOpenHelper checks if database exists
- If not exists → onCreate() is called

- If version changes → onUpgrade() is called
- Database is opened for operations

Example Code (Full Class)
Java

```
public class MyDatabaseHelper extends SQLiteOpenHelper {

    public MyDatabaseHelper(Context context) {
        super(context, "StudentDB", null, 1);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE TABLE Student(id INTEGER PRIMARY KEY, name TEXT)");
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        db.execSQL("DROP TABLE IF EXISTS Student");
        onCreate(db);
    }
}
```

Advantages
- Easy database management
- Reduces coding complexity
- Handles database creation automatically
- Manages version control
- Prevents manual errors

Disadvantages
- Suitable only for small/medium applications
- Requires SQL knowledge
- Not ideal for large cloud-based systems

Q4. Explain CRUD Operations (Insert, Read, Update, Delete) of SQLite Database

Introduction
In Android, SQLite Database is used for storing structured data locally. To manage data in a database, we perform CRUD operations, which are the basic operations for any database system. It is widely used in applications developed using Android Studio.

What is CRUD?

CRUD stands for:
C → Create (Insert)
R → Read (Retrieve)
U → Update
D → Delete
These operations are used to manage records in SQLite database tables.

CRUD Operations in SQLite

1. Create (Insert Operation)

- Used to add new data (record) into the table.

Java

```
ContentValues values = new ContentValues();
values.put("id", 1);
values.put("name", "Rahul");
db.insert("Student", null, values);
```

2. Read (Retrieve Operation)

- Used to fetch data from database.

Java

```
Cursor cursor = db.rawQuery("SELECT * FROM Student", null);
while(cursor.moveToNext()) {
   String name = cursor.getString(1);
}
```

3. Update Operation

- Used to modify existing data in table.

Java

```
ContentValues values = new ContentValues();
values.put("name", "Amit");
db.update("Student", values, "id=?", new String[]{"1"});
```

4. Delete Operation

- Used to remove data from table.

Java

```
db.delete("Student", "id=?", new String[]{"1"});
```

Working of CRUD Operations

1. App connects to SQLite database
2. User performs action (insert/read/update/delete)
3. SQL query is executed
4. Data is stored or retrieved from table

Advantages of CRUD Operations

- Easy data management
- Full control over database records
- Efficient local storage handling
- Useful for offline applications

Q5. Define Firebase Realtime Database. Explain setup and connection with Android Studio

Introduction

In modern Android applications, cloud storage is very important for storing and syncing data online. One of the most popular cloud databases is Firebase Realtime Database, provided by Google. It is widely used in mobile applications developed using Android Studio.

Definition

Firebase Realtime Database is a cloud-based NoSQL database that stores data in JSON format and allows real-time synchronization of data across all connected devices.

Features of Firebase Realtime Database

- Real-time data synchronization
- Cloud-based database (no local server required)
- NoSQL structure (JSON tree format)
- Cross-platform support (Android, iOS, Web)
- Offline data support (syncs when internet returns)
- Secure access using authentication rules

Advantages

- Easy setup and integration
- No server management required
- Fast real-time updates
- Scalable and reliable
- Suitable for live applications

Setup Steps in Android Studio

Step 1: Create Firebase Project

Open Firebase Console
Click on Add Project
Enter project name and continue

Step 2: Add Android App
Register your Android application
Enter package name of app
Download google-services.json file

Step 3: Add google-services.json
Place file inside app/ directory in Android project

Step 4: Add Firebase Dependencies
Add in Gradle file:
Gradle
implementation 'com.google.firebase:firebase-database:latest-version'

Also apply plugin:
Gradle
apply plugin: 'com.google.gms.google-services'

Step 5: Sync Project
- Sync Gradle files
- Firebase will be connected successfully

Connection with Android Studio
After setup, connect Firebase database in Java code:

Java
DatabaseReference databaseReference =
FirebaseDatabase.getInstance().getReference("Users");
This creates a reference to the Users node in Firebase database.

Working of Firebase Realtime Database
1. App connects to Firebase cloud
2. Data is stored in JSON tree structure
3. Any change in data is updated in real-time
4. All connected users get updated instantly

Use Cases
- Chat applications (WhatsApp-like apps)
- Live tracking apps
- Online gaming leaderboards
- Real-time form submissions
- Social media apps

Q6. How to Insert, Retrieve, and Display Data using Firebase

Introduction
In modern Android applications, cloud databases are widely used to store and manage data online. Firebase Realtime Database, provided by Google, allows developers to insert, retrieve, and display data in real-time. It is commonly used in applications developed using Android Studio.

Firebase Database Reference
Before performing any operation, we need to create a reference to the database:

Java
```
DatabaseReference databaseReference =
      FirebaseDatabase.getInstance().getReference("Users");
```
This creates a reference to the Users node in Firebase database.

Inserting Data into Firebase
Data is inserted using the setValue() method.

Java

```
databaseReference.child("1").setValue("Rahul");
```

Explanation
"Users" → Main node
"1" → Child node (unique ID)
"Rahul" → Value stored

You can also insert objects:
Java

```
User user = new User("Rahul", 20);
databaseReference.child("1").setValue(user);
```

Retrieving Data from Firebase
Method 1: Using get() (One-time read)
Java
```
databaseReference.child("1").get().addOnSuccessListener(snapshot -> {
   String name = snapshot.getValue(String.class);
});
```

Method 2: Using ValueEventListener (Real-time)
Java
```
databaseReference.addValueEventListener(new ValueEventListener() {
   @Override
```

```
    public void onDataChange(DataSnapshot snapshot) {
        for (DataSnapshot data : snapshot.getChildren()) {
            String value = data.getValue(String.class);
        }
    }

    @Override
    public void onCancelled(DatabaseError error) {
    }
});
```

Displaying Data in UI

After retrieving data, it can be displayed using UI components like:

- TextView → for single data
- ListView / RecyclerView → for multiple data

Example

Java

```
textView.setText(name);
```

Working of Firebase Data Handling

1. App connects to Firebase cloud
2. Data is inserted using setValue()
3. Data is stored in JSON tree format
4. Data is retrieved using listeners
5. UI is updated automatically

Advantages

- Real-time data synchronization
- Easy to use and implement
- No need of backend server
- Supports multiple users simultaneously
- Offline support with automatic sync

Disadvantages

- Requires internet connection (mostly)
- Limited querying compared to SQL
- Data structure must be well designed

Use Cases

- Chat applications
- Live tracking apps
- Online forms
- Real-time dashboards

UNIT 5

Q1. Write a short note on Kotlin for Android

Introduction
Kotlin is a modern, statically-typed programming language used for Android app development. It is officially supported by Google and is fully integrated with Android Studio.

Definition
Kotlin is a programming language designed to be concise, safe, and fully interoperable with Java, making it ideal for Android development.

Features of Kotlin
- Simple and clean syntax
- Fully compatible with Java
- Null safety (prevents null pointer exceptions)
- Less boilerplate code
- Supports object-oriented and functional programming
- Smart type inference

Advantages of Kotlin
- Reduces code length
- Improves readability
- Fewer errors due to null safety
- Faster development
- Easy maintenance

Example Code
Kotlin

```
fun main() {
    val name = "Rahul"
    println("Hello $name")
}
```

Use in Android
- Used to develop Android apps
- Replace Java in modern development
- Supported by all Android libraries

Q2. Overview of Cross-platform Mobile Application Development: Flutter and React Native

Introduction
In mobile app development, creating separate applications for Android and iOS can be time-consuming and costly. To solve this problem, Cross-platform Mobile Application Development is used. It allows developers to build applications that run on multiple platforms using a single codebase. This approach is widely used in modern app development.

What is Cross-platform Development?
Cross-platform development is a method in which a developer writes code once and uses it on multiple platforms like Android and iOS.

Need for Cross-platform Development

- Reduces development time
- Saves cost
- Simplifies maintenance
- Ensures consistency across platforms

Flutter
Introduction

- Flutter is an open-source UI toolkit developed by Google for building high-performance mobile applications.

Features of Flutter

- Uses Dart programming language
- Provides hot reload feature
- Rich set of customizable widgets
- High performance (near native speed)
- Supports Android, iOS, Web, and Desktop

Advantages of Flutter

- Single codebase for multiple platforms
- Fast development with hot reload
- Attractive and customizable UI
- High performance
- Disadvantages of Flutter
- Large app size
- Requires learning Dart language

React Native
Introduction

- React Native is a cross-platform framework developed by Meta Platforms that allows developers to build mobile apps using JavaScript.

Features of React Native

- Uses JavaScript and React
- Uses native components
- Code reusability
- Strong community support
- Supports Android and iOS

Advantages of React Native
- Easy for web developers
- Faster development
- Reusable components
- Large community support

Disadvantages of React Native
- Performance is slightly lower than Flutter
- Depends on third-party libraries

Advantages of Cross-platform Development
- Write once, run anywhere
- Saves time and cost
- Faster deployment
- Easier maintenance
- Consistent user experience

Use Cases
- E-commerce apps
- Social media apps
- Business applications
- Startup projects

Q3. Write a short note on APIs and JSON Parsing

Introduction
In modern Android applications, most apps need to communicate with online servers to fetch or send data such as user information, weather updates, news, etc. This communication is done using APIs, and the data is usually transferred in JSON format. These concepts are widely used in apps developed using Android Studio.

API (Application Programming Interface)
Definition
- An API is a set of rules and protocols that allows one application to interact and communicate with another application or server.

Types of APIs
1. Web API – Used over the internet (HTTP/HTTPS)

2. REST API – Most commonly used API in mobile apps
3. Library API – Used within software libraries

Working of API

- User performs an action in the app
- App sends a request to the server via API
- Server processes the request
- Server sends response (data) back
- App displays the data to the user

Example Use Cases

- Weather applications
- Online shopping apps
- Social media apps
- Payment systems

JSON (JavaScript Object Notation)

Definition

- JSON is a lightweight data format used for data exchange between client and server.

Features of JSON

- Simple and easy to understand
- Lightweight and fast
- Uses key-value pairs
- Platform independent
- Widely supported

Example JSON Data

JSON

```
{
  "name": "Rahul",
  "age": 20,
  "city": "Ahmedabad"
}
```

JSON Parsing

Definition

- JSON Parsing is the process of converting JSON data into usable objects or variables in an Android application.

Methods of JSON Parsing in Android

- Using JSONObject
- Using JSONArray

- Using libraries like Gson or Retrofit

Example Code
Java

```
JSONObject jsonObject = new JSONObject(jsonString);

String name = jsonObject.getString("name");
int age = jsonObject.getInt("age");
String city = jsonObject.getString("city");
```

Advantages of APIs and JSON Parsing
- Enables communication between app and server
- Efficient data transfer
- Easy to use and implement
- Makes apps dynamic and real-time
- Supports cross-platform applications

Disadvantages
- Requires internet connection
- Security issues if not handled properly
- Dependency on server availability

Q4. Give the Overview of Publishing an Android App

Introduction
After developing an Android application, the final step is to make it available to users. This process is called publishing an Android app. The most popular platform for publishing Android apps is the Google Play Store. Developers usually build and prepare apps using Android Studio.

Definition
Publishing an Android app is the process of preparing, uploading, testing, and releasing an application on an app distribution platform so that users can download and install it.

Requirements for Publishing
Before publishing, the following things are required:
- Completed Android application
- Signed APK or AAB file
- Developer account on Google Play Console
- App icon and screenshots
- App description and category
- Privacy policy

Steps to Publish an Android App

Step 1: Create Developer Account

- Register on Google Play Console
- Pay one-time registration fee
- Fill required details

Step 2: Prepare the Application

- Test the app thoroughly
- Remove bugs and errors
- Optimize performance and UI
- Ensure app meets Play Store policies

Step 3: Generate Signed APK/AAB

- Use Android Studio to generate signed APK/AAB
- Required for secure publishing

Step 4: Create App Listing

- Add app name
- Write description
- Upload screenshots and icon
- Select category (Education, Games, etc.)

Step 5: Upload Application

- Upload APK or AAB file
- Add version details

Step 6: Set Pricing and Distribution

- Choose Free or Paid app
- Select countries where app will be available

Step 7: Content Rating & Policies

- Fill content rating questionnaire
- Follow Google Play policies
- Provide privacy policy

Step 8: Submit for Review

- Submit app for review
- Google reviews app for quality and security

Step 9: Publish App

- After approval, app is published
- Users can download from Play Store

Post-Publishing Activities

- Monitor user feedback and ratings
- Fix bugs and release updates
- Add new features

Advantages of Publishing
- Global reach of users
- Opportunity to earn money
- Easy distribution and updates
- Builds developer reputation

Important Points
- Follow all Play Store guidelines
- Ensure app security and privacy
- Maintain app quality
- Provide regular updates

Q5. List Recent Trends and Technologies in Android App Development

Introduction
Android app development is rapidly evolving with new technologies and trends. These advancements help developers build modern, fast, scalable, and user-friendly applications. Most of these technologies are widely used in apps developed using Android Studio and supported by Google.

Recent Trends and Technologies

1. Kotlin Programming Language
   - Official language for Android
   - Concise and safe compared to Java
   - Reduces code length and errors

2. Cross-platform Development
   - Frameworks like Flutter and React Native
   - Single codebase for Android and iOS
   - Saves time and development cost

3. Firebase Services
   - Provides backend services
   - Real-time database, authentication, cloud storage
   - Easy integration with Android apps

4. Artificial Intelligence (AI) & Machine Learning (ML)
   - Used for smart features
   - Face recognition, chatbots, recommendation systems

- Improves user experience

5. Internet of Things (IoT)
   - Connects mobile apps with smart devices
   - Used in smart homes, healthcare, and automation

6. Cloud-based Applications
   - Data stored on cloud servers
   - Access data from anywhere
   - Improves scalability and performance

7. Augmented Reality (AR)
   - Combines real-world and virtual objects
   - Used in gaming, shopping, education apps

8. 5G Technology
   - High-speed internet connectivity
   - Better app performance
   - Faster data transfer and streaming

9. Jetpack Libraries
   - Set of Android libraries provided by Google
   - Helps in UI design, navigation, lifecycle management
   - Simplifies development

10. Progressive Web Apps (PWA)
    - Web applications that behave like mobile apps
    - Lightweight and fast
    - No need to install from app store

Advantages of These Technologies
- Faster development process
- Improved app performance
- Better user experience
- Easy maintenance and scalability
- Supports modern app features